AUDIT TECHNIQUE

Diagnostic complet codebase & organisation

Prepare pour **Twoghether** · Fevrier 2026 · Confidentiel

0%	6.4x	44%	36
COUVERTURE TESTS	RATIO INEFFICIENCE	PRESENCE DEV	MOIS DE PROJET

En 30 secondes

Together est un produit **techniquement ambitieux et fonctionnel** : marketplace CESU/URSSAF, Stripe Connect, messagerie temps reel, KYC Dataleon. L'architecture backend est solide (Clean Architecture + DDD).

Le probleme n'est pas le developpeur. C'est l'absence de structure : pas de PM pour prioriser, pas de CTO pour arbitrer, specs inexistantes, presence fractionnee (44%). Yoann travaille a l'aveugle dans un systeme qui genere de la dette mecaniquement.

Risques immediats : faille de securite axios (CVE), Laravel 10 EOL depuis fevrier 2025, CRA abandonne. **Recommandation** : audit de 2 jours pour structurer les priorites avant d'investir davantage.

Sommaire

01 Constats Organisation →

02 Diagnostic Technique →

03 Analyse Temps & Equipe →

04 Prise de Hauteur →

05 Prochaine Etape →

06 Scenarios d'accompagnement →

1. Constats Organisation

CONSTAT A

Melange de niveaux de granularite

Taches dev, actions commerciales et decisions strategiques cohabitent dans la meme vue. Aucune separation entre la roadmap produit/tech et le plan commercial.

CONSTAT B

Un « cahier des charges » qui n'en est pas un

Le CDC est un journal de notes brutes : retours utilisateurs, idees, bugs, questions ouvertes melanges. Aucun statut formel, aucune priorite, aucune assignation.

CONSTAT C

Pas de priorisation formelle

Colonne « Importance » vide sur la quasi-totalite des lignes. La mention « V2 » sert de fourre-tout pour reporter des sujets sans date cible.

> *Le developpeur travaille a l'aveugle, en piochant dans un flux de demandes non triees. Ce n'est pas un probleme de competences. C'est l'absence d'une couche PM entre la vision et l'execution.*

2. Diagnostic Technique

Stack : **React 18 + Laravel 10 + Socket.io**. Environ 63K lignes de code (32K TypeScript, 31K PHP).

Qu'est-ce que cette stack ?

React = bibliotheque JavaScript pour construire l'interface utilisateur (ce que voit le client). **Laravel** = framework PHP qui gere toute la logique metier cote serveur (paiements, reservations, etc.). **Socket.io** = permet la messagerie en temps reel entre utilisateurs.

CRITERE	ETAT	CRITICITE
Architecture backend	Clean Architecture + DDD	SOLIDE
Design Patterns	Action pattern + ACL	BON
Qualite code	Variable selon zones	MOYEN
Securite	45 vulnerabilites (1 critique)	CRITIQUE
Stack frontend	CRA abandonne (EOL)	CRITIQUE
Stack backend	Laravel 10 EOL fev. 2025	CRITIQUE
Tests	0 fichiers sur 906	CRITIQUE
CI/CD	Inexistant	CRITIQUE

Traduction des termes techniques

metier (creer une reservation, valider un paiement) est isolee dans sa propre classe. **ACL** = systeme de permissions qui controle qui peut faire quoi. **CI/CD** = systeme automatique qui verifie le code et le deploie. Son absence = tout est manuel et risque.

Dependances critiques

axios	`^0.22.0`	Faille de securite critique CVE-2025-27152
react-scripts (CRA)	`^5.0.1`	Projet abandonne par Facebook — migration requise
moment.js	`^2.29.4`	Obsolete (67KB) — remplacer par dayjs (2KB)
laravel/framework	`^10.0`	Fin de support fevrier 2025

Librairies manquantes

State Management

Pas de Redux ou Zustand. L'etat de l'application est disperse = bugs de synchronisation.

Data Fetching

Pas de TanStack Query. Le cache est gere manuellement = incoherences de donnees.

Analyse statique

Pas de PHPStan. Les erreurs de type ne sont detectees qu'en production.

Points forts

- Architecture backend bien pensee : le code metier est isole et reutilisable
- Deploiement automatise via Deployer (deploy.php)
- Sentry configure pour le monitoring d'erreurs en production

3. Analyse Temps & Equipe

Le projet a ete developpe par deux equipes distinctes sur 36 mois. Cette donnee change completement l'interpretation du diagnostic.

Agence externe

Fev. 2023 → Oct. 2024 (20 mois)

1045	39.5%	5
Commits	Bug ratio	Developpeurs

Yoann (interne)

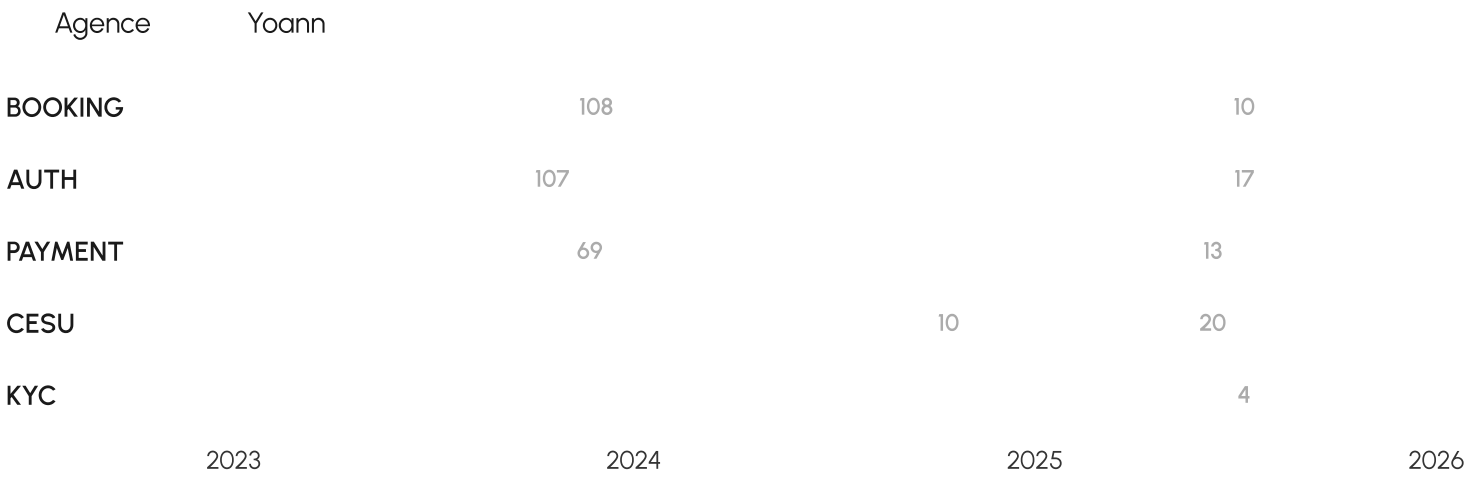
Nov. 2024 → Jan. 2026 (15 mois)

249	35.3%	44%
Commits	Bug ratio	Presence

Qu'est-ce que le « bug ratio » ?

C'est le pourcentage de commits qui corrigent des bugs plutot qu'ajoutent des fonctionnalites. Un ratio eleve indique qu'on passe plus de temps a reparer qu'a construire. 35% est acceptable pour un projet en maintenance, mais Yoann devrait pouvoir construire plus s'il avait des specs claires.

Timeline par module



Repartition par Epic

AUTH	107	17	42%	OAuth, sessions, inscription CESU
BOOKING	108	10	38%	Reservations, disponibilites, calendrier
SERVICES	81	19	35%	Catalogue, categories, pricing, filtres
PAYMENT	69	13	55%	Stripe Connect, factures — zone instable
MESSAGING	73	7	40%	Socket.io, notifications temps reel
ADMIN	65	—	61%	Backoffice complet — dette elevee
CESU/URSSAF	10	20	33%	Integration URSSAF, CESU+, AICI
KYC	—	4	25%	Dataleon v1 + v2 — from scratch
ADS/B2B	—	12	17%	Publicites, fonctionnalites B2B — from scratch

▪ Agence | ▪ Yoann. Les modules crees from scratch par Yoann ont un bug ratio 2x meilleur (17-25% vs 40-61%).

Metriques globales du projet

500

Commits bug/fix

41% du total — presque 1 commit sur 2 corrige quelque chose.

14

Commits refactoring

1% du total — le code n'est jamais nettoye, la dette s'accumule.

91

31%

sans consolidation.

1/3 du code est reecrit.

Chronologie detaillee

Phase Agence (Fev 2023 → Oct 2024)

Fev-Avr 2023 — Setup initial, Auth, structure DB

Mai-Juil 2023 — MVP Booking + Services + Messaging

Juil 2023 — Pic bugs 74.8% (stabilisation post-MVP)

Aout-Oct 2023 — Payment Stripe Connect, Admin

Nov 2023-Jan 2024 — Refonte UI, optimisations

Fev-Oct 2024 — Maintenance, fixes, handover

Phase Yoann (Nov 2024 → Jan 2026)

Nov-Dec 2024 — Prise en main, config CESU

Jan 2025 — KYC Dataleon from scratch

Fev-Mar 2025 — CESU workflow complet

Avr-Juin 2025 — Features contest/decline, fixes

Juil-Aout 2025 — Pause estivale (10% presence)

Sep 2025-Jan 2026 — Filtres, Dataleon v2, B2B

Ce que revele l'analyse

La dette vient de l'agence

Les modules PAYMENT (55% bugs) et ADMIN (61% bugs) ont ete livres sous pression avec une dette technique importante.

Yoann performe mieux

35.3% de bug ratio vs 39.5% pour l'agence. Sur ses creations propres (KYC, ADS), son ratio descend a 17-25%.

Presence tres variable

Janvier 83%, Fevrier 15%, Juin 14%, Aout 10%. Impossible de maintenir un rythme avec ces variations.

7 gaps inexpliques

Periodes de 5+ jours sans commits hors vacances. Contexte perdu a chaque reprise.

METRIQUE	VALEUR	INTERPRETATION
Temps reel consomme	**~566 jours**	Agence + Yoann cumules
Temps optimal estime	**~88 jours**	Senior focalise, specs claires, bonne archi
Ratio d'inefficience	**6.4x**	On a mis 6x plus de temps que necessaire
Jours avec activite	**298 jours**	Sur 36 mois (~780 jours ouvres) = 38% d'activite

> **Caveat sur les metriques de presence**
>
> Ces chiffres sont bases uniquement sur ce repo. Yoann travaille peut-etre sur d'autres projets ou repos non analyses. Le taux de 44% ne reflete pas forcement son temps de travail total — seulement sa disponibilite sur Together.

D'ou vient cette inefficience ?

Context switching = changer de tache constamment coute cher cognitivement. Chaque interruption = 20-30 min pour retrouver le fil. **Absence de specs** = le dev doit deviner ce qu'on veut, fait des allers-retours, recommence. **Dette technique** = sans tests, chaque modification peut casser autre chose, donc on avance prudemment. **Presence fractionnee** (44%) = on perd le contexte entre chaque session de travail.

4. Prise de Hauteur

> *Ce projet n'a pas un probleme de developpeur. Il a un probleme de structure.*

Together est un produit ambitieux et fonctionnel : marketplace de services a la personne avec integration CESU/URSSAF, paiement Stripe Connect, messagerie temps reel, KYC automatise via Dataleon. C'est techniquement complexe — et ca fonctionne.

L'agence a livre un MVP avec une bonne architecture backend. Le choix de Clean Architecture + DDD etait judicieux pour un produit qui devra evoluer. Mais la pression de livraison a cree de la dette : 0 tests, dependances figees, zones instables.

Yoann a herite de cette base. Depuis 15 mois, il travaille seul, a 44% de presence, sans specs formalisees. Malgre cela, ses metriques sont meilleures que celles de l'agence. Il a cree le module KYC/Dataleon from scratch, refait le systeme de publicites, finalise le B2B.

Profil estime de Yoann

Niveau

Mid-level (3-5 ans). Maitrise Laravel/React, applique les patterns existants. Execute bien quand les specs sont claires.

Forces

Autonome, fiable. Ses creations from scratch (KYC, ADS) sont plus stables que le code herite. Sait s'adapter a une archi existante.

Limites — Tests

Limites — Proactivite tech

Dans les deux cas, c'est un manque de cadre.

escale les alertes securite. Attend qu'on lui dise.

Limites — IA & outils modernes

Aucune trace d'utilisation d'outils IA (Copilot, ChatGPT) dans son workflow. Pas de CI/CD mis en place. Travaille « a l'ancienne ».

Verdict

Bon executant dans un systeme dysfonctionnel. Avec un PM, un cadre clair et de l'outillage moderne, sa velocite doublerait.

Le vrai probleme

Pas de PM

Aucune priorisation. Yoann pioche dans un flux de demandes non triees. Il ne sait pas ce qui compte vraiment.

Pas de CTO

Personne pour arbitrer tech vs produit. La migration Laravel 11 ? Les tests ? La securite ? Tout est reporte indefiniment.

Presence fractionnee

44% de presence = context switching permanent. Le cout cognitif est enorme. La velocite s'effondre mecaniquement.

Dette invisible

0 tests = pas de filet de securite. Chaque modification peut tout casser. La peur de toucher au code s'installe.

5. Prochaine Etape

PHASE	OBJECTIF
Jour 1 - Matin	Atelier vision : objectifs court/moyen/long terme, documentation des priorites par pilier.
Jour 1 - Apres-midi	Restructuration backlog, priorisation MoSCoW, definition des workflows de developpement.
Jour 2 - Matin	Audit technique approfondi : analyse module par module, cartographie de la dette.
Jour 2 - Apres-midi	Restitution : diagnostic complet + roadmap avec jalons et points de decision.

Livrables

Rapport complet

Diagnostic organisation + technique, analyse par module, cartographie de la dette, risques priorises.

Backlog restructure

Separation bugs/evolutions/idees/dette technique. Priorisation MoSCoW. Estimation de charge.

Plan securite immediat

Patch axios CVE, upgrade dependances critiques, mise en place CI/CD minimal.

Roadmap 3-6-12 mois

Vision avec jalons, dependances, et points de decision pour chaque scenario.

6. Scenarios d'accompagnement

A l'issue de l'audit, plusieurs trajectoires possibles selon vos priorites business.

Scenario A : Stabilisation urgente — Court terme

Patch des vulnerabilites critiques, mise en place CI/CD minimal, tests sur zones sensibles (PAYMENT, AUTH). Accompagnement ponctuel de Yoann.

Scenario B : Reprise Front + Mobile — Moyen terme

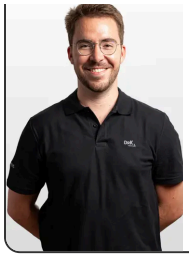
Le backend Clean Architecture est solide. Le front CRA est le point faible. Migration vers Vite/Next.js, refonte progressive de l'app mobile. Yoann reste sur le backend, Drakkar prend le front.

Scenario C : Modernisation complete — Long terme

Upgrade Laravel 11+, migration TypeScript 5, mise en place TanStack Query + Zustand, architecture de tests (80% coverage zones critiques), monitoring.

Scenario D : CTO / CPO fractionne — Structurel

Drakkar comme CTO ou CPO externalise. Pilotage du developpeur, structuration backlog, arbitrages techniques, reviews de code. 1-2 jours/semaine pour installer une vraie gouvernance produit/tech.

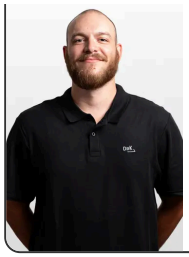


Nathan Menard

CEO

Strategie · IA · Direction technique

nathan.menard@drakkar.io



Matthis Ruel

Responsable Studio

Product management · Design · Startups

matthis.ruel@drakkar.io

Thibault Piqueras

Account Manager

Conseil dirigeants & transformation digitale

thibault.piqueras@drakkar.io

DRAKKAR GROUP

Fevrier 2026